A Execution of the Minimal Counterexample

This appendix gives the detailed execution traces underlying the counterexample of Section 4.3. It first analyzes the minimal counterexample for NegamaxTTM, then compares it with the corresponding execution of NegamaxTTW. Both algorithms are called with identical inputs:

$$\texttt{NegamaxTT[W,M]}(v, \alpha = 0, \beta = 5, \mathit{depth} = 2, T),$$

where $T = \{v \mapsto (\mathit{value} = 3, \mathit{depth} = 4, \mathit{flag} = \mathsf{LB})\}$.

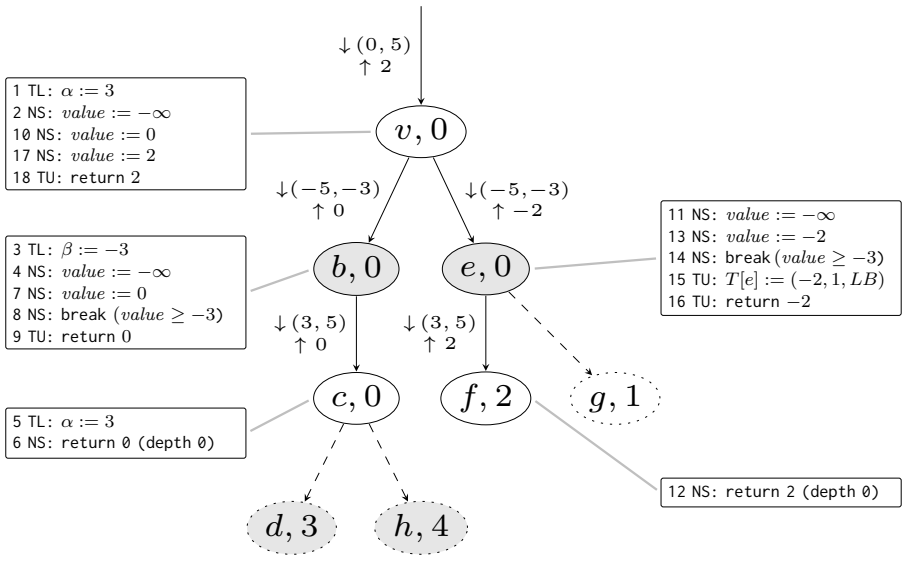

Fig. 4: Execution trace of NegamaxTTM on the minimal counterexample from Figure 3(c), with transposition table $\{v \mapsto (\mathit{value} = 3, \mathit{depth} = 4, \mathit{flag} = \mathsf{LB})\}$. Nodes contain an identifier and an evaluation value. Rectangular blocks show statements executed at each node, labeled by algorithmic phase (TL/NS/TU) and global execution order. Edge labels show the alpha-beta interval passed downward (↓) and the value returned upward (↑).

A.1 NegamaxTTM

Figure 4 shows the recursive depth-first search of NegamaxTTM through nodes v, b, c, e, and f. For each node, a text block lists the relevant statements executed during its search, prefixed with their global execution order. The statements are labeled by phase: **TL** (Table Lookup), **NS** (Negamax Search), and **TU** (Table Update). Edges are labeled with two pieces of information:

- $\downarrow (\alpha, \beta)$: the alpha-beta window passed to the child, after the standard negation and swapping of bounds $(-\beta, -\alpha)$,
- $\uparrow$ *value*: the value returned by the child.

Dashed edges and nodes (e.g., d, g, h) are not visited during this search. The key steps that lead to the problematic return value 2 are:

1. At the root v, the table lookup increases α from 0 to 3 (step 1). This narrowing of the window is caused by the stored lower bound for v.
2. When searching child b, the window becomes $(-5, -3)$ after negation. Its child c returns its own value 0 (step 6), since it is searched at depth 0. The search of b propagates this value 0 (step 9) to the root v (step 10).
3. Child e is searched with window $(-5, -3)$. Its leaf child f returns 2, which becomes -2 after negation (step 13). Since $-2 \geq -3$, a beta-cutoff occurs (step 14), skipping the better child g with value 1. The value -2 is stored in the TT (step 15) and propagated back to the root v (step 16). After negation (step 17) the final return value becomes 2.

The stored lower bound 3 for v thus causes a premature cutoff at e. Consequently, the algorithm never visits the subtree that would reveal the better minimax value 1 available at depth 2. The return value 2 cannot be justified by any witness expansion of $\lfloor v \rfloor_2$, confirming the violation of our correctness criterion.

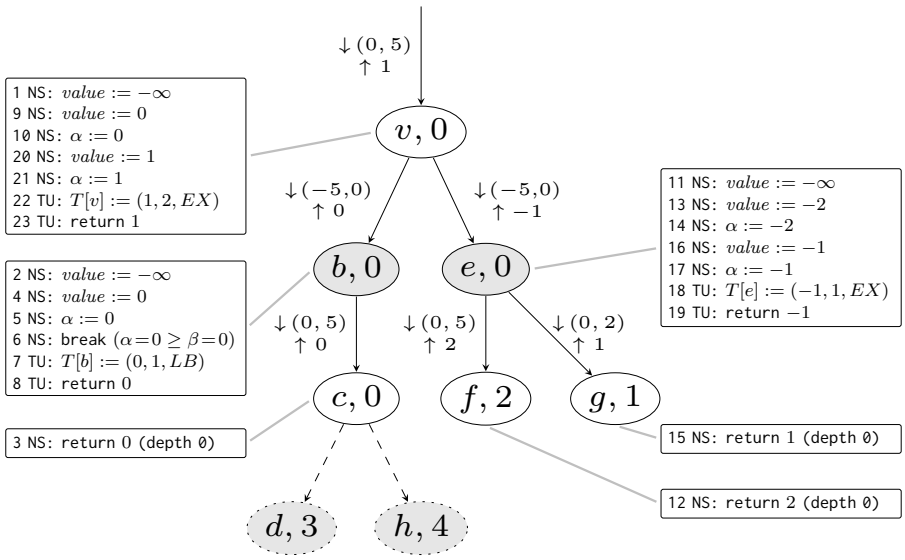

Fig. 5: Execution trace of NegamaxTTW on the minimal counterexample from Figure 3(c), with transposition table $\{v \mapsto (value = 3, depth = 4, flag = \mathsf{LB})\}$.

A.2 NegamaxTTW

Figure 5 illustrates the execution of NegamaxTTW. The table lookup at v also finds the stored lower bound 3, but the check $t.value \geq \beta$ (i.e., $3 \geq 5$) fails, so the lookup does *not* cause an early return. Consequently, the search proceeds with the original window $(0, 5)$, explores all relevant children, and returns the expected negamax value 1. The stored entry 3 is effectively ignored because it does not guarantee a cutoff in the current, wider window.

A.3 Comparison

The two traces highlight the semantic difference between the table-lookup strategies. NegamaxTTM uses stored bounds to narrow the search window, which can lead to incorrect pruning when a bound computed under a narrow window is reused in a wider one. NegamaxTTW only accepts a stored value if it immediately guarantees a cutoff; otherwise it discards the entry and performs a full search. This more conservative policy preserves the existence of a witness tree, explaining why NegamaxTTW satisfies our correctness criterion while NegamaxTTM does not.